SECURITY ASSESSMENT

Vulnerability Assessment Report

Target: https://www.shivnerpratishthan.org/

ASSESSMENT TYPE	Passive, Non-Intrusive Security Review
ASSESSMENT DATE	04 September 2026
CLASSIFICATION	Confidential — For Authorized Recipient Only
SCOPE	Public-facing website (passive analysis only)

1. Executive Summary

This report documents a **passive, non-destructive** security assessment of the public-facing website `https://www.shivnerpratishthan.org/`, conducted under an authorized engagement scope.

Important context for this engagement: the assessment was performed using an **automated content-fetching and OSINT toolset**, not a raw-socket/browser-based scanning environment. This means some standard passive checks (raw HTTP response headers, TLS/certificate parameters, cookie attributes, DNS/hosting records, and live network request inspection) **could not be technically executed** from this platform — outbound raw HTTP/TLS connections to the target host were blocked by the assessment environment's own network policy (confirmed during testing — see Section 3, Limitations).

Per the assessment instructions, **no findings have been invented**. Where a check could not be completed, it is explicitly labeled as **"Unable to Verify"** rather than reported as a confirmed vulnerability. This report separates:

- **Confirmed Observations** — things directly evidenced during this assessment, and
- **Items Requiring Direct Verification** — standard checks that need to be run with unrestricted tooling (commands/tools provided in Section 7) to reach a complete conclusion.

Headline observations:

- The target resolves and serves content over **HTTPS** without any TLS handshake failure being surfaced to the fetch client (weak positive signal only — not equivalent to a full TLS/certificate audit).
- The homepage returned **minimal server-rendered HTML** to an automated, non-JavaScript-executing fetch, consistent with a client-side-rendered (JavaScript-heavy) site architecture. This is primarily an architectural/SEO observation, not a vulnerability in itself.
- **No technology fingerprint, server banner, or exposed metadata** could be extracted from the content that was returned.
- **All header-, cookie-, and TLS-specific checks requested in scope are marked "Unable to Verify"** in this report and require follow-up with the tools listed in Section 7 to close out.

This should be treated as a **partial assessment / reconnaissance summary**, not a completed header and configuration audit. Section 7 provides the exact commands to finish the remaining checks.

---

2. Scope

- Authorized assessment only
- Public-facing website: `https://www.shivnerpratishthan.org/`
- Passive security analysis only
- No destructive testing
- No data modification
- No unauthorized exploitation, brute forcing, or injection attempts were performed

---

3. Methodology

The assessment followed a passive reconnaissance and configuration-review approach aligned to the requested structure:

- **1. Reconnaissance** — Fetched the homepage via an automated content-retrieval tool; searched public/indexed sources for OSINT (organizational registration data, indexed subpages, cached technical fingerprints).
- **2. Security Configuration Review** — Attempted to evaluate HTTPS usage and the presence of standard security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy).
- **3. Browser-Layer Analysis** — Attempted to review cookies, local storage, and network request behavior.
- **4. OWASP-Aligned Risk Review** — Reviewed available evidence for Security Misconfiguration, Information Disclosure, Authentication weaknesses, Access Control issues, and Injection risk indicators.

Limitations encountered (disclosed transparently):

- The assessment environment's outbound network policy blocks raw HTTP/TCP connections to arbitrary third-party hosts (confirmed via a direct connection attempt, which returned a `host_not_allowed` policy response rather than data from the target).
- The content-fetch tool available in this environment returns **parsed page content only**, not raw HTTP response headers, TLS handshake details, or `Set-Cookie` values.
- No headless-browser/DevTools access was available, so **local storage, session storage, and the live network request waterfall could not be inspected**.
- No DNS/WHOIS/IP-intelligence tool was available to identify hosting provider or ASN with certainty.

Because of this, **Sections 5 and 6 clearly separate confirmed findings from unverifiable items**, and Section 7 gives you (or your engineer) the exact commands/tools to complete the remaining checks. If you run those and share the output, this report can be finalized with full severity ratings.

---

4. Tools Used

Tool	Purpose	Outcome
Automated HTTPS content fetcher	Retrieve rendered page content from the target URL	Succeeded — minimal content returned
Web-based OSINT / search	Identify indexed pages, cached fingerprints, organizational registration data	No technical fingerprint data found
Direct outbound HTTP request (`curl`)	Attempt raw header retrieval	**Blocked by sandbox network policy** (`host_not_allowed`) — confirms no raw header data was accessible in this environment

Tools recommended to complete this assessment (see Section 7 for exact commands): `curl -I`, `openssl s_client`, securityheaders.com (https://securityheaders.com), SSL Labs SSL Test (https://www.ssllabs.com/ssltest/), Mozilla Observatory, browser DevTools (Application/Network tabs), Wappalyzer/BuiltWith.

---

5. Findings Summary Table

#	Finding Name	Category	Status	Severity
F-1	HTTPS reachable, no handshake failure on fetch	Transport Security	Confirmed (weak signal)	Informational
F-2	Minimal server-rendered HTML / likely client-rendered architecture	Information Disclosure / Architecture	Confirmed	Informational
F-3	No technology stack / server banner identifiable from available data	Information Disclosure	Confirmed (absence of evidence)	Informational
F-4	Security response headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)	Security Misconfiguration	**Unable to Verify**	Not Rated
F-5	Cookie attributes (Secure, HttpOnly, SameSite)	Session Security	**Unable to Verify**	Not Rated
F-6	Local/session storage contents	Client-Side Data Exposure	**Unable to Verify**	Not Rated
F-7	Network request behavior / third-party calls	Data Flow / Privacy	**Unable to Verify**	Not Rated
F-8	TLS configuration (protocol versions, cipher suites, certificate chain)	Transport Security	**Unable to Verify**	Not Rated
F-9	Authentication surfaces, access control, injection points	OWASP Risk Categories	**No login/input surfaces observed in available content — not applicable to date**	Not Rated

6. Detailed Findings

F-1: HTTPS Reachable Without Client-Side Handshake Failure

Severity: Informational

Description: The target homepage was retrievable over `https://www.shivnerpratishthan.org/` without a TLS validation error being raised by the fetching client.

Technical Explanation: Modern fetch clients reject connections on expired, self-signed, or mismatched certificates. No such rejection occurred here.

Evidence: Successful `HTTP 200`-equivalent content retrieval via `https://www.shivnerpratishthan.org/`; `final_url` remained on the `https://` scheme with no forced downgrade to HTTP observed.

Business Impact: Baseline transport encryption appears to be in place. This is a positive, low-information signal only — it does **not** confirm certificate validity period, key strength, protocol version (e.g., whether legacy TLS 1.0/1.1 is still enabled), or cipher suite strength.

Recommendation: Run a dedicated TLS assessment (Section 7) to confirm protocol versions in use, certificate expiry, and cipher suite strength.

F-2: Minimal Server-Rendered HTML Content

Severity: Informational

Description: A passive, non-JavaScript-executing fetch of the homepage returned only a page title (`Shivner Pratishthan`) and negligible additional markup.

Technical Explanation: This pattern is typical of client-side-rendered applications (e.g., React/Vue/Angular single-page apps) where content is injected into the DOM by JavaScript after the initial HTML shell loads, rather than being present in the raw server response.

Evidence: Two independent fetch attempts (different extraction methods) of `https://www.shivnerpratishthan.org/` both returned only the page title and a stray HTML comment marker, with no body content, navigation, or text extracted.

Business Impact: Primarily an architectural and SEO/accessibility consideration (search engine crawlers and some accessibility tools that don't execute JavaScript may see an empty page). It is **not** inherently a security vulnerability, but it does mean this assessment could not visually or structurally inspect page content, forms, or embedded scripts through the automated fetch — a manual browser-based review is needed for full coverage (see Section 7).

Recommendation: Confirm via browser DevTools that critical content/functionality does not silently fail for users with JavaScript disabled or blocked, and that no sensitive logic is exposed client-side unnecessarily.

F-3: No Technology Fingerprint Identifiable From Available Data

Severity: Informational

Description: No CMS signature, JavaScript framework marker, server banner, or generator meta tag was identifiable from the content returned to this assessment's tooling.

Technical Explanation: This is an **absence-of-evidence** observation, not a statement that no fingerprintable information exists — the tooling available could not retrieve `Server`/`X-Powered-By` headers or fully rendered DOM/script sources.

Evidence: No generator meta tags, script `src` references, or server headers were present in the retrieved content.

Business Impact: Reduced technology fingerprint visibility is mildly favorable from a reconnaissance-resistance perspective, but this should not be treated as confirmed — a proper scan (Section 7) may reveal a fingerprintable stack.

Recommendation: Run Wappalyzer/BuiltWith and a header check to determine actual technology exposure, then assess whether version numbers are being disclosed unnecessarily.

F-4 through F-8: Unable to Verify — Requires Direct Tooling

The following checks were **explicitly in scope** but could not be technically completed from this assessment environment, because outbound raw HTTP/TCP connections to the target host were blocked by this platform's own network policy (confirmed via a direct connection attempt that returned a policy-level `host_not_allowed` response rather than any data from the target), and no headless-browser/DevTools access was available.

Per the engagement instructions, **these are reported as open items requiring verification — not as confirmed vulnerabilities**:

Item	What's Needed	Why It Matters
F-4: Security response headers (`Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`)	Raw HTTP response header capture	Missing CSP/X-Frame-Options can enable clickjacking; missing HSTS can allow protocol-downgrade attacks; missing X-Content-Type-Options can enable MIME-sniffing issues
F-5: Cookie attributes (`Secure`, `HttpOnly`, `SameSite`)	Raw `Set-Cookie` header capture or DevTools Application tab	Missing attributes can expose session tokens to XSS or network interception
F-6: Local/session storage	Browser DevTools Application tab	Sensitive data (tokens, PII) should not be stored unencrypted client-side
F-7: Network request behavior	Browser DevTools Network tab	Identifies third-party calls, mixed-content (HTTP resources on an HTTPS page), and unexpected data flows
F-8: TLS configuration	`openssl s_client` or SSL Labs scan	Confirms protocol versions, cipher strength, and certificate validity/chain

F-9: OWASP-Aligned Risk Categories — Authentication, Access Control, Injection

Status: No login form, admin panel, search field, or other user-input surface was present in the content retrieved during this assessment, so **no injection, authentication, or access-control testing surface was identified to evaluate** — and none was tested, per the passive/no-exploitation scope in any case.

Recommendation: If the site has a contact form, donation/payment flow, admin login, or any other input/authentication surface not surfaced during this limited fetch, it should be explicitly reconnoitered (passively — e.g., reviewing form field names, autocomplete attributes, and whether submission occurs over HTTPS) and added to this report as a follow-up.

---

7. Recommendations

To complete the header, cookie, and TLS checks in scope, run the following (safe, passive, read-only) commands and share the output for full analysis:

```
# 1. Full response headers over HTTPS
curl -sI https://www.shivnerpratishthan.org/

# 2. Check whether HTTP is redirected to HTTPS (and how)
curl -sI http://www.shivnerpratishthan.org/
```

```
# 3. Inspect the TLS certificate and protocol
openssl s_client -connect www.shivnerpratishthan.org:443 -servername
www.shivnerpratishthan.org </dev/null 2>/dev/null | openssl x509 -noout -dates -issuer
-subject

# 4. Check for common exposed paths (passive, read-only)
curl -sI https://www.shivnerpratishthan.org/robots.txt
curl -sI https://www.shivnerpratishthan.org/sitemap.xml
curl -sI https://www.shivnerpratishthan.org/.well-known/security.txt
```

Online tools (no installation required):

- Header/config check: https://securityheaders.com
- TLS/certificate check: https://www.ssllabs.com/ssltest/
- Overall security posture: https://observatory.mozilla.org
- Technology fingerprint: Wappalyzer browser extension

General hardening recommendations (standard best practice, applicable regardless of current configuration — confirm against actual results):

- Enforce HTTPS with an `HTTP` → `HTTPS` redirect and enable `Strict-Transport-Security` with a reasonable `max-age` (e.g., 6–12 months) once confirmed all subdomains support HTTPS.
- Set a baseline `Content-Security-Policy` to mitigate XSS/clickjacking, even a permissive starting policy (`default-src 'self'`) that is iteratively tightened.
- Set `X-Content-Type-Options: nosniff` and either `X-Frame-Options: DENY/SAMEORIGIN` or a `frame-ancestors` CSP directive.
- Set `Referrer-Policy` (e.g., `strict-origin-when-cross-origin`) to limit referrer leakage to third parties.
- Set a `Permissions-Policy` to disable unused browser features (camera, microphone, geolocation, etc.) by default.
- If any cookies are set, ensure `Secure`, `HttpOnly`, and an appropriate `SameSite` value are applied.
- Periodically re-run a technology fingerprint scan and patch/update any identified CMS, plugins, or libraries to current versions.
- If a contact/donation form exists, confirm it submits over HTTPS and passes serverside validation (not testable passively without further scope).

---

8. Conclusion

Within the constraints of this passive, tooling-limited assessment, no confirmed vulnerabilities were identified — but this reflects **limited verification coverage**, not a clean bill of health. The site is reachable over HTTPS with no immediate TLS handshake failures, and it does not appear to leak an obvious technology fingerprint through the content retrievable by this assessment's tooling.

However, **six scoped checks (security headers, cookie attributes, local storage, network requests, and full TLS configuration) could not be completed** due to environment network restrictions and are the priority next step. Running the commands in Section 7 and sharing the output will allow this report to be finalized with accurate, evidence-based severity ratings for every in-scope item, in line with the "no invented findings" requirement of this engagement.

This report reflects a point-in-time, passive assessment with disclosed tooling limitations (Section 3). It should not be represented as a complete penetration test or a comprehensive security audit. Re-testing is recommended after any remediation and periodically thereafter (e.g., quarterly or after significant site changes).